

DATA 260 Homework 3 Report

GITHUB REPOSITORY: <https://github.com/rutujaste06/data260-5825>

Student ID- 020065825

SID4- 5825

PORT_BASE - 8425

PREFIX - s5825

SEED - 5825

VERIFY_SEED – 265825

DOMAIN_ID - 1

DOMAIN - Clinical trial listings

HARDWARE - MacBook Pro, Apple M5 chip

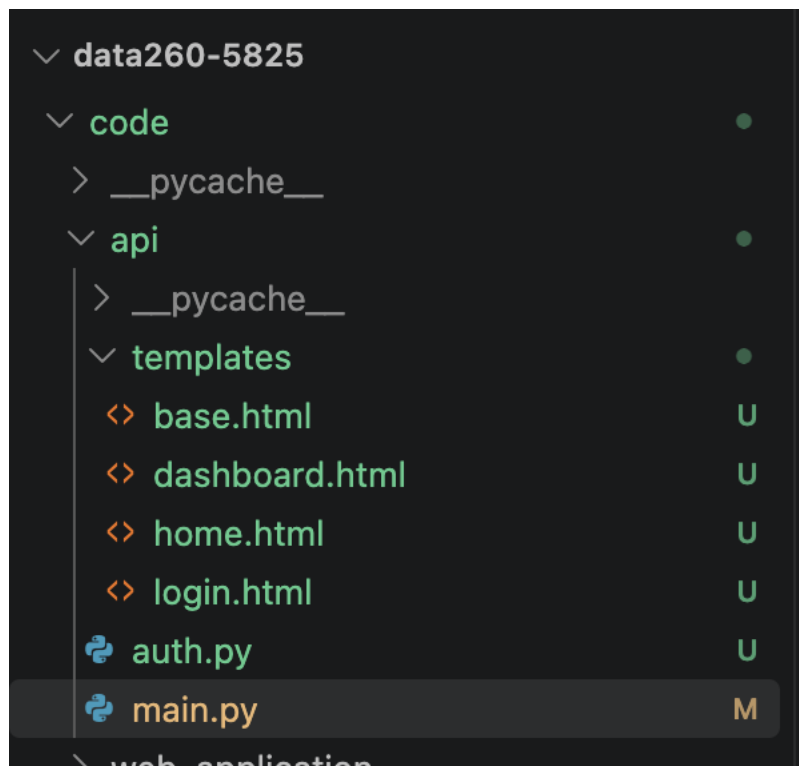
LOCAL MODEL USED: qwen3:8b (Ollama)

TAGGED COMMIT HASH: b6e8261d384a7c9bf2f555aae7f38480417dc124

Outputs:

Created the Fast API application to connect the LOGIN and LOGOUT webpages with the backend

Templates directory including all the html pages:



Started the FastAPI application:

```
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825/code/api
uvicorn main:app --reload --port 8425
INFO: Will watch for changes in these directories: ['/Users/rutujasaste/Documents/data260-5825/code/api']
INFO: Uvicorn running on http://127.0.0.1:8425 (Press CTRL+C to quit)
INFO: Started reloader process [17651] using StatReload
INFO: Started server process [17653]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

```
INFO: 127.0.0.1:62726 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:62730 - "GET /login HTTP/1.1" 200 OK
INFO: 127.0.0.1:62731 - "POST /login HTTP/1.1" 200 OK
```

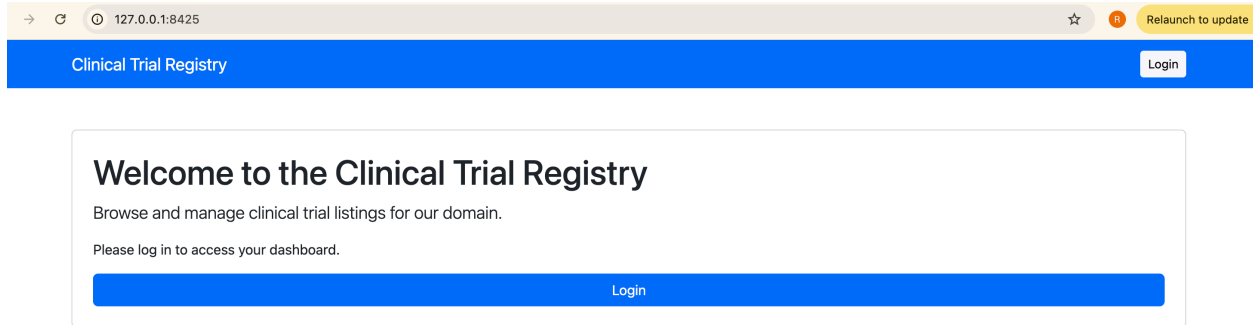
Homepage:

Code:

```
15
16 @router.get("/", response_class=HTMLResponse)
17 def home(request: Request):
18     user = request.session.get("user")
19     return templates.TemplateResponse(
20         request, "home.html", {"user": user}
21     )
22
23
```

```
{% if user %}
    <p>You're logged in as <strong>{{ user }}</strong>.</p>
    <a class="btn btn-primary" href="/dashboard">Go to dashboard</a>
    <a class="btn btn-outline-secondary" href="/logout">Log out</a>
{% else %}
    <p>Please log in to access your dashboard.</p>
    <a class="btn btn-primary" href="/login">Login</a>
{% endif %}
```

Output that shows the homepage:



Login page:

Code:

```
@router.get("/login", response_class=HTMLResponse)
def login_form(request: Request):
    return templates.TemplateResponse(
        request, "login.html", {"error": None}
    )
```

```
<form method="post" action="/login">
  <div class="mb-3">
    <label for="username" class="form-label">Username</label>
    <input type="text" class="form-control" id="username" name="username" required>
  </div>
  <div class="mb-3">
    <label for="password" class="form-label">Password</label>
    <input type="password" class="form-control" id="password" name="password" required>
  </div>
  <button type="submit" class="btn btn-primary w-100">Log in</button>
</form>
```

Output showing the login form:

→ 127.0.0.1:8425/login ☆ Relaunch to update

Clinical Trial Registry Login

Login

Username

Password

Login

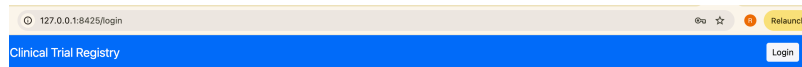
Adding wrong password that shows red alert and doesnot allow to login:

Code:

```
@router.post("/login", response_class=HTMLResponse)
def login_submit(request: Request, username: str = Form(...), password: str = Form(...)):
    if USERS.get(username) == password:
        # Save who's logged in inside the signed session cookie
        request.session["user"] = username
        return RedirectResponse(url="/dashboard", status_code=302)
    # Wrong credentials -> show Bootstrap alert on the same page
    return templates.TemplateResponse(
        request,
        "login.html",
        {"error": "Invalid username or password."},
    )
```

```
{% if error %}
    <div class="alert alert-danger" role="alert">
        {{ error }}
    </div>
{% endif %}
```

Output: Entering wrong password that is not predefined



Login

Invalid username or password.

Username

Password

Log in

Logging in with the hardcoded login details

```
# Hardcoded demo users |
USERS = {
    "admin": "password123",
    "researcher1": "trial2026",
    "coordinator": "sjsu2026",
}
```

```
@router.get("/dashboard", response_class=HTMLResponse)
def dashboard(request: Request):
    user = request.session.get("user")
    if not user:
        return RedirectResponse(url="/login", status_code=302)
    response = templates.TemplateResponse(
        request, "dashboard.html", {"user": user}
    )

    response.headers["Cache-Control"] = "no-store, no-cache, must-revalidate"
    response.headers["Pragma"] = "no-cache"
    return response
```

```
<div class="card p-4">
    <h1>Welcome, {{ user }}</h1>
    <p class="lead">This is your protected dashboard. Only logged-in users can see this page.</p>
    <a class="btn btn-outline-secondary" href="/logout">Log out</a>
</div>
```

Logged in as an admin



Welcome, admin

This is your protected dashboard. Only logged-in users can see this page.

Log out

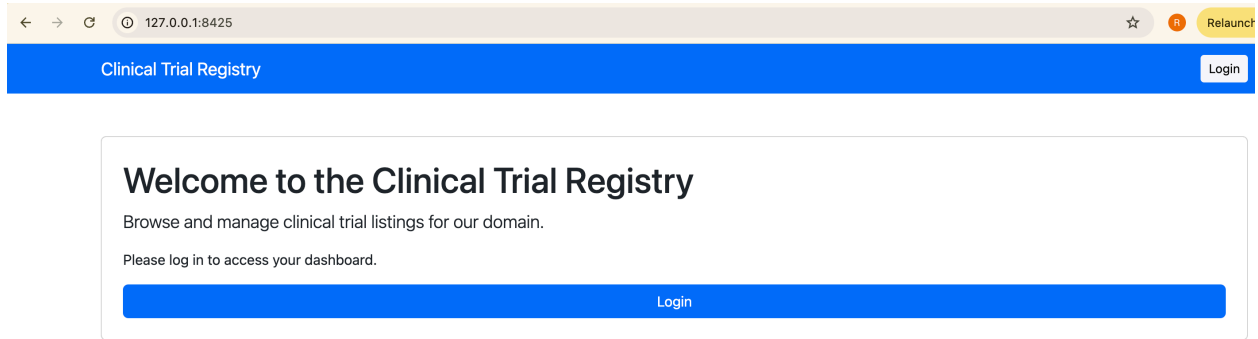
Validated that an expired or logged-out session cannot be reused to reach /dashboard.

Code:

```
if not user:
    return RedirectResponse(url="/login", status_code=302)
```

```
@router.get("/logout")
def logout(request: Request):
    request.session.clear()
    return RedirectResponse(url="/", status_code=302)
```

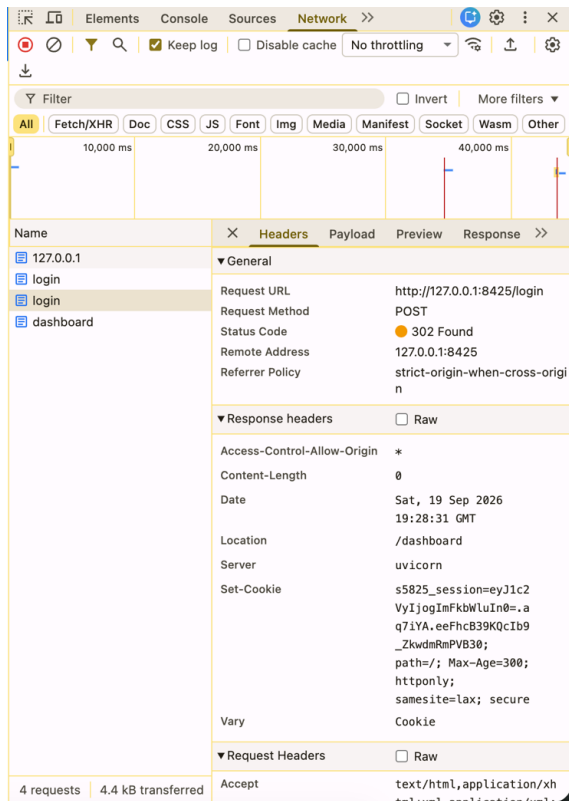
After 300 secs automatically the page gets logged out:



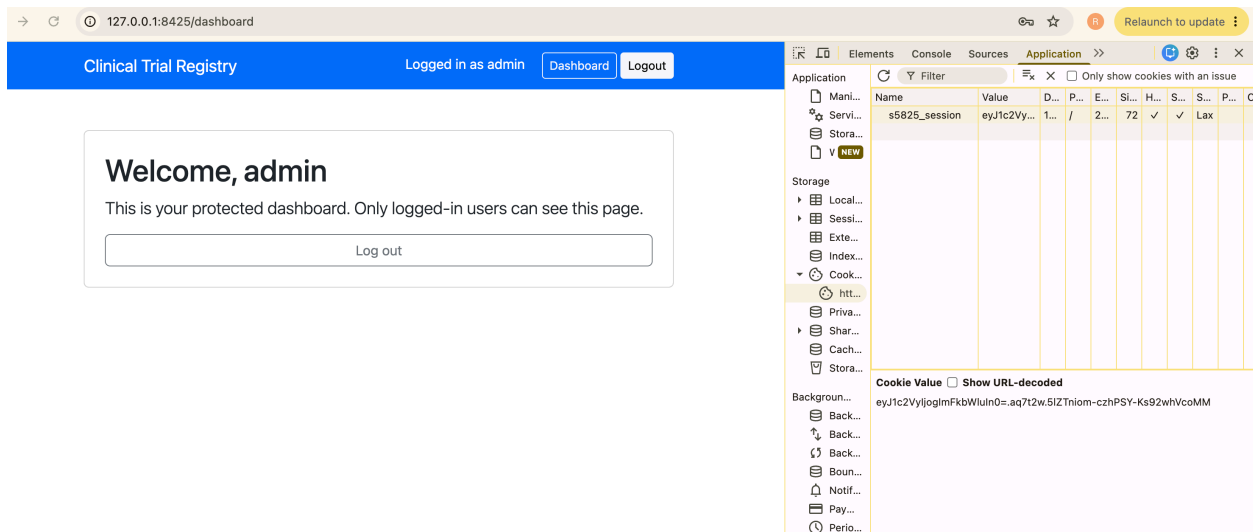
Navbar used for navigation and other bootstrap components:

```
<nav class="navbar navbar-expand-lg navbar-dark bg-primary">
  <div class="container">
    <a class="navbar-brand" href="/">Clinical Trial Registry</a>
    <div>
      {% if user %}
        <span class="navbar-text text-white me-3">Logged in as {{ user }}</span>
        <a class="btn btn-outline-light btn-sm" href="/dashboard">Dashboard</a>
        <a class="btn btn-light btn-sm" href="/logout">Logout</a>
      {% else %}
        <a class="btn btn-light btn-sm" href="/login">Login</a>
      {% endif %}
    </div>
  </div>
</nav>
```

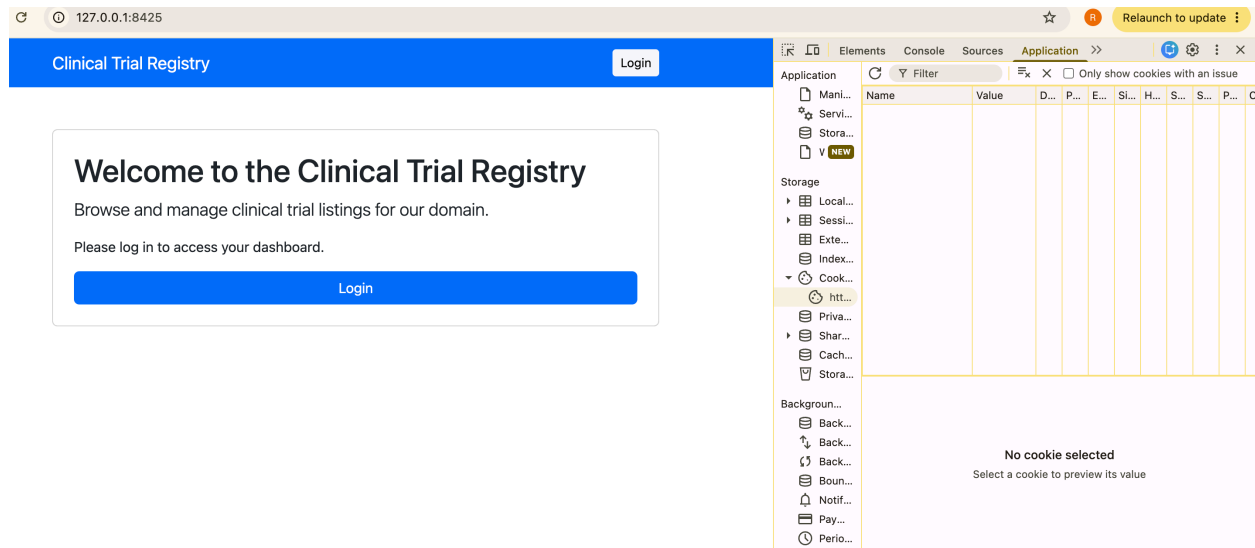
Validated that the session cookie is secure by checking the Set-Cookie response header showing all three attributes:



Session cookie when logged in as a admin



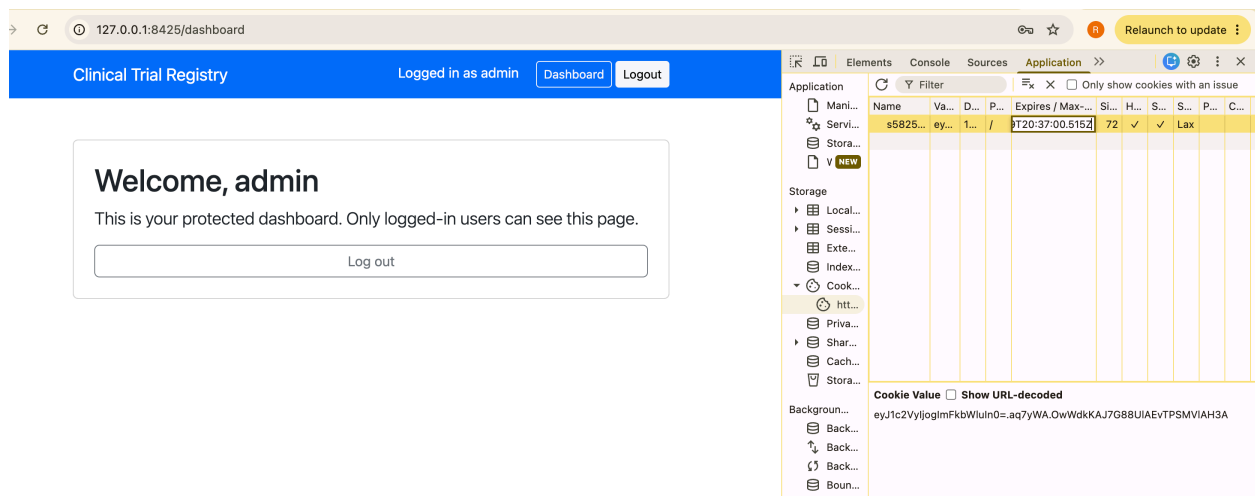
Session cookie cleared after logging out



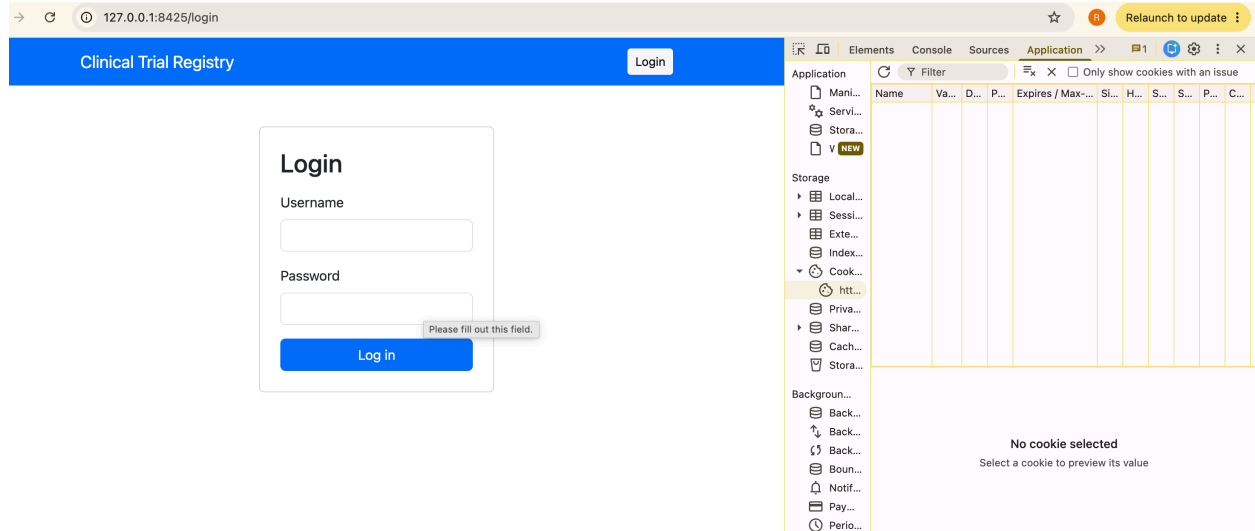
Validated the idle session timeout by changing the “max_age to 20” for testing

```
# Session/cookie support for the login system
app.add_middleware(
    SessionMiddleware,
    secret_key="s5825-dev-secret-change-later",
    session_cookie="s5825_session",
    max_age=20,
    https_only=True,
)
```

Logging in :



After 20secs the session timed out and after refreshing the page the page logged out: Dashboard is not accessible after the timeout and it lands to the login page:



Auth.py code :

```
from fastapi import APIRouter, Request, Form
from fastapi.responses import RedirectResponse, HTMLResponse
from fastapi.templating import Jinja2Templates

router = APIRouter()
templates = Jinja2Templates(directory="templates")

# Hardcoded demo users
USERS = {
    "admin": "password123",
    "researcher1": "trial2026",
    "coordinator": "sjsu2026",
}

@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    user = request.session.get("user")
    return templates.TemplateResponse(
        request, "home.html", {"user": user}
    )

@router.get("/login", response_class=HTMLResponse)
def login_form(request: Request):
    return templates.TemplateResponse(
        request, "login.html", {"error": None}
    )

@router.post("/login", response_class=HTMLResponse)
```

```

def login_submit(request: Request, username: str = Form(...), password: str = Form(...)):
    if USERS.get(username) == password:
        # Save who's logged in inside the signed session cookie
        request.session["user"] = username
        return RedirectResponse(url="/dashboard", status_code=302)
    # Wrong credentials -> show Bootstrap alert on the same page
    return templates.TemplateResponse(
        request,
        "login.html",
        {"error": "Invalid username or password."},
    )

@router.get("/dashboard", response_class=HTMLResponse)
def dashboard(request: Request):
    user = request.session.get("user")
    if not user:
        # No valid session (never logged in, logged out, or expired) -> kick back to login
        return RedirectResponse(url="/login", status_code=302)
    response = templates.TemplateResponse(
        request, "dashboard.html", {"user": user}
    )
    # Force the browser to always re-check with the server instead of
    # showing a cached copy when the user hits Back after logging out
    response.headers["Cache-Control"] = "no-store, no-cache, must-revalidate"
    response.headers["Pragma"] = "no-cache"
    return response

@router.get("/logout")
def logout(request: Request):
    request.session.clear() # destroys the session -> old cookie becomes useless
    return RedirectResponse(url="/", status_code=302)

```

PART 2:

A: Environment setup:

Corpus fetch (SOURCES.md + CORPUS_MANIFEST.json):

Fetches 55 studies- 217.9 KB

```
def fetch_studies():
    #Pull raw study records from the ClinicalTrials.gov v2 API
    params = {
        "query.cond": CONDITION,
        "pageSize": PAGE_SIZE,
        "fields": ",".join([
            "NCTId",
            "BriefTitle",
            "OfficialTitle",
            "OverallStatus",
            "Phase",
            "Condition",
            "BriefSummary",
            "DetailedDescription",
            "EligibilityCriteria",
            "LeadSponsorName",
        ]),
    }
    resp = requests.get(API_URL, params=params, timeout=30)
    resp.raise_for_status()
    return resp.json().get("studies", [])
```

```
• (venv) rutujaste@Rutujas-MacBook-Pro data260-5825 % python code/corpus/fetch_corpus.py
  Fetched 55 studies from ClinicalTrials.gov
  Saved 55 files to data/corpus/
  Total corpus size: 217.9 KB (OK, above 200KB minimum)
  Wrote reports/hw03/SOURCES.md
  Wrote reports/hw03/CORPUS_MANIFEST.json
```

Corpus + embedding model load:

```
def get_embed_model():
    return HuggingFaceEmbedding(model_name=EMBED_MODEL_NAME)

def load_documents():
    reader = SimpleDirectoryReader(input_dir=CORPUS_DIR)
    documents = reader.load_data()
    return documents
```

Embedding proof:

Committed questions.yaml file first:

To satisfy the requirement that `questions.yaml` be committed before running any retrieval, I committed this file first, before building or running any of the three chunking pipelines. This creates a permanent, timestamped record in my git history proving my five expected answers were written down before I saw any retrieval output, so they could not be adjusted afterward to match the results.

Commit:

```
git commit -m "Add questions.yaml for retrieval comparison"
```

[main b48ff07] Add questions.yaml for retrieval comparison

Committer: Rutuja Saste <rutujasaste@Rutujas-MacBook-Pro.local>

```
git config --global --edit
```

```
git commit --amend --reset-author
```

1 file changed, 30 insertions(+)

```
create mode 100644 reports/hw03/questions.yaml
```

Token chunking:

Code:

Output:

Semantic chunking :

Output:

Code:

Output:

Full comparison run:

```
for name, module in TECHNIQUES.items():
    rows, summary = run_technique(name, module, questions, embed_model)
    all_rows_by_technique[name] = rows
    summaries.append(summary)
    chunk_stats[name] = {
        "num_chunks": summary["num_chunks"],
        "avg_chunk_len": summary["avg_chunk_len"],
    }

save_raw(all_rows_by_technique, chunk_stats)
write_metrics_md(summaries)
```

showing all 5 questions with 3 techniques

```
(venv) rutujasast@Rutujas-MacBook-Pro data260-5825 % mkdir -p reports/hw03
python code/rag/run_comparison.py | tee reports/hw03/RUN_LOG.txt
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% | 103/103 [00:00<00:00, 10414.48it/s]
Loading weights: 100% | 103/103 [00:00<00:00, 12373.64it/s]
Loaded 5 questions from reports/hw03/questions.yaml

=====
Building index for: token
=====
token: 258 chunks, avg length 935.0 chars
q1: top1_cosine=0.8224 mean@3_cosine=0.7731 recall_hit=True latency=8.44ms
q2: top1_cosine=0.8100 mean@3_cosine=0.6128 recall_hit=True latency=4.86ms
q3: top1_cosine=0.6228 mean@3_cosine=0.6050 recall_hit=True latency=4.92ms
q4: top1_cosine=0.6897 mean@3_cosine=0.5949 recall_hit=True latency=5.03ms
q5: top1_cosine=0.5319 mean@3_cosine=0.5371 recall_hit=False latency=4.75ms

=====
Building index for: semantic
=====
Loading weights: 100% | 103/103 [00:00<00:00, 17084.40it/s]
semantic: 135 chunks, avg length 1650.8 chars
q1: top1_cosine=0.7696 mean@3_cosine=0.6748 recall_hit=True latency=4.29ms
q2: top1_cosine=0.8028 mean@3_cosine=0.6088 recall_hit=True latency=3.96ms
q3: top1_cosine=0.6996 mean@3_cosine=0.6105 recall_hit=True latency=4.20ms
q4: top1_cosine=0.7131 mean@3_cosine=0.5925 recall_hit=True latency=4.01ms
q5: top1_cosine=0.5883 mean@3_cosine=0.5646 recall_hit=True latency=4.09ms

=====
Building index for: sentence_window
=====
sentence_window: 1063 chunks, avg length 209.7 chars
q1: top1_cosine=0.7674 mean@3_cosine=0.7291 recall_hit=True latency=13.01ms
q2: top1_cosine=0.8475 mean@3_cosine=0.7673 recall_hit=True latency=9.52ms
q3: top1_cosine=0.6408 mean@3_cosine=0.6238 recall_hit=True latency=9.79ms
q4: top1_cosine=0.7209 mean@3_cosine=0.6743 recall_hit=True latency=9.53ms
q5: top1_cosine=0.5858 mean@3_cosine=0.5856 recall_hit=True latency=9.57ms
Wrote reports/hw03/raw/retrieval_token.csv
Wrote reports/hw03/raw/retrieval_semantic.csv
Wrote reports/hw03/raw/retrieval_sentence_window.csv
Wrote reports/hw03/raw/retrieval_all.json
Wrote reports/hw03/METRICS.md

=====
DONE. Summary:
=====
{'technique': 'token', 'num_chunks': 258, 'avg_chunk_len': 935.0, 'avg_top1_cosine': 0.6953741613734217, 'avg_mean_at_k_cosine': 0.6245903252079429, 'recall_at_k': 0.8, 'avg_latency_ms': 5.598708393517882}
{'technique': 'semantic', 'num_chunks': 135, 'avg_chunk_len': 1650.8, 'avg_top1_cosine': 0.7146877068901141, 'avg_mean_at_k_cosine': 0.6102211602960848, 'recall_at_k': 1.0, 'avg_latency_ms': 4.111392004415393}
{'technique': 'sentence_window', 'num_chunks': 1063, 'avg_chunk_len': 209.65004703668862, 'avg_top1_cosine': 0.7124710853581261, 'avg_mean_at_k_cosine': 0.676024727063114, 'recall_at_k': 1.0, 'avg_latency_ms': 10.285299989409}
```

Confidently-wrong retrieval verification:

```
Loading weights: 100% | 103/103 [00:00<00:00, 31375.79it/s]
=== RANK 3 chunk (NCT02722239) ===
NCT ID: NCT02722239
Brief Title: An Open-label, Randomized, Crossover Study of Comparative Pharmacokinetics and Bioequivalence of Dapagliflozin + Metformin, 10 mg + 1000 mg Versus the Combined Use of Forxiga™, 10 mg and Two Glucophage® Long, ER Tablets, 500 mg Co-administered to Healthy Volunteers Under Standard Fed Conditions
Official Title: An Open-label, Randomized, Crossover Study of Comparative Pharmacokinetics and Bioequivalence of Dapagliflozin + Metformin Modified Release Film-coated Tablets, 10 mg + 1000 mg (AstraZeneca AB, Sweden) Versus the Combined Use of Forxiga™ (Dapagliflozin), Film-coated Tablets, 10 mg (Bristol Myers Squibb Company, USA) and Two Glucophage® Long (Metformin), ER Tablets, 500 mg (Merck Santé S.A.S., France), Co-administered to Healthy Volunteers Under Standard Fed
=== RANK 1 chunk (NCT05816057) ===
NCT ID: NCT05816057
Brief Title: A Study of the Efficacy and Safety of Semaglutide Injection in the Treatment of Type 2 Diabetes Mellitus
Official Title: A Multicenter, Randomized, Open-label, and Parallel Assignment Phase III Study to Compare the Efficacy and Safety of Semaglutide Injection With Ozempic® in Combination With Metformin in the Treatment of Type 2 Diabetes
Status: COMPLETED
Phase: PHASE3
Conditions: Diabetes Mellitus, Type 2
Sponsor: Hangzhou Jiuyuan Gene Engineering Co. Ltd.,

Brief Summary:
Semaglutide injection is a new drug developed according to Ozempic® biosimilars. This trial is conducted in China. The purpose of this study is to investigate the similarities in the efficacy and safety of semaglutide injection and Ozempic® in the treatment of type 2 diabetes, respectively.

Detailed Description:
This study is a multicenter, randomized, open-label, parallel assignment, which plans to enroll 476 patients with type 2 diabetes And Stable Contains Ozempic: True
```

Output:


```

Loading weights: 100%| 103/103 [00:00<00:00, 31375.79it/s]
=== RANK 3 chunk (NCT02722239) ===
NCT ID: NCT02722239
Brief Title: An Open-label, Randomized, Crossover Study of Comparative Pharmacokinetics and Bioequivalence of Dapagliflozin + Metformin, 10 mg + 1000 mg Versus the Combined Use of Forxiga™, 10 mg and Two Glucophage® Long, ER Tablets, 500 mg Co-administered to Healthy Volunteers Under Standard Fed Conditions
Official Title: An Open-label, Randomized, Crossover Study of Comparative Pharmacokinetics and Bioequivalence of Dapagliflozin + Metformin Modified Release Film-coated Tablets, 10 mg + 1000 mg (AstraZeneca AB, Sweden) Versus the Combined Use of Forxiga™ (Dapagliflozin), Film-coated Tablets, 10 mg (Bristol Myers Squibb Company, USA) and Two Glucophage® Long (Metformin), ER Tablets, 500 mg (Merck Santé S.A.S., France), Co-administered to Healthy Volunteers Under Standard Fed
=== RANK 1 chunk (NCT05816057) ===
NCT ID: NCT05816057
Brief Title: A Study of the Efficacy and Safety of Semaglutide Injection in the Treatment of Type 2 Diabetes Mellitus
Official Title: A Multicenter, Randomized, Open-label, and Parallel Assignment Phase III Study to Compare the Efficacy and Safety of Semaglutide Injection With Ozempic® in Combination With Metformin in the Treatment of Type 2 Diabetes
Status: COMPLETED
Phase: PHASE3
Conditions: Diabetes Mellitus, Type 2
Sponsor: Hangzhou Jiuyuan Gene Engineering Co. Ltd.,

Brief Summary:
Semaglutide injection is a new drug developed according to Ozempic® biosimilars. This trial is conducted in China. The purpose of this study is to investigate the similarities in the efficacy and safety of semaglutide injection and Ozempic® in the treatment of type 2 diabetes, respectively.

Detailed Description:
This study is a multicenter, randomized, open-label, parallel assignment, which plans to enroll 476 patients with type 2 diabetes And Stable Contains Ozempic: True

```

What to compare:

```

def run_technique(name, module, questions, embed_model):
    print(f"\n{'=' * 60}\nBuilding index for: {name}\n{'=' * 60}")
    index, nodes = module.build_index()
    num_chunks = len(nodes)
    avg_chunk_len = sum(len(n.get_content()) for n in nodes) / num_chunks
    print(f"{name}: {num_chunks} chunks, avg length {avg_chunk_len:.1f} chars")

    all_rows = []
    per_query_stats = []

    for q in questions:
        query = q["question"]
        expected_source = q.get("expected_source", "")

        query_embedding = embed_model.get_query_embedding(query)

        start = time.perf_counter()
        retriever = index.as_retriever(similarity_top_k=K)
        results = retriever.retrieve(query)
        latency_ms = (time.perf_counter() - start) * 1000

        cosine_sims = []
        found_expected = False

        for rank, node_with_score in enumerate(results, start=1):
            node = node_with_score.node
            chunk_text = node.get_content()
            store_score = node_with_score.score
            file_name = node.metadata.get("file_name", "")

            if expected_source and expected_source in file_name:
                found_expected = True

            chunk_embedding = embed_model.get_text_embedding(chunk_text)
            cosine_sim = cosine_similarity(query_embedding, chunk_embedding)
            cosine_sims.append(cosine_sim)

        all_rows.append({
            "technique": name,
            "question_id": q["id"],
            "query": query,
            "rank": rank,
            "store_score": round(store_score, 4) if store_score is not None else None,
            "cosine_sim": round(cosine_sim, 4),
            "chunk_len": len(chunk_text),

```

```

1         file_name = file_name,
2         "preview": chunk_text[:160].replace("\n", " "),
3         "latency_ms": round(latency_ms, 2),
4     })
5
6
7     top1_cosine = cosine_sims[0] if cosine_sims else 0.0
8     mean_at_k_cosine = sum(cosine_sims) / len(cosine_sims) if cosine_sims else 0.0
9
10    per_query_stats.append({
11        "question_id": q["id"],
12        "top1_cosine": top1_cosine,
13        "mean_at_k_cosine": mean_at_k_cosine,
14        "latency_ms": latency_ms,
15        "recall_hit": found_expected,
16    })
17
18    print(f" {q['id']}: top1_cosine={top1_cosine:.4f} mean@{K}_cosine={mean_at_k_cosine:.4f} "
19          f"recall_hit={found_expected} latency={latency_ms:.2f}ms")
20
21    summary = {
22        "technique": name,
23        "num_chunks": num_chunks,
24        "avg_chunk_len": avg_chunk_len,
25        "avg_top1_cosine": sum(s["top1_cosine"] for s in per_query_stats) / len(per_query_stats),
26        "avg_mean_at_k_cosine": sum(s["mean_at_k_cosine"] for s in per_query_stats) / len(per_query_stats),
27        "recall_at_k": sum(1 for s in per_query_stats if s["recall_hit"]) / len(per_query_stats),
28        "avg_latency_ms": sum(s["latency_ms"] for s in per_query_stats) / len(per_query_stats),
29    }
30
31    return all_rows, summary

```

Technique	Chunks	Avg chunk length	Top-1 cosine	Mean@k cosine	Recall@k	Mean retrieval latency (ms)
Token	258	935.0	0.6954	0.6246	0.80	5.60
Semantic	135	1650.8	0.7147	0.6102	1.00	4.11
Sentence-window	1063	209.7	0.7125	0.6760	1.00	10.29

The values in this table were generated by `run_comparison.py`, which ran all 5 questions from `questions.yaml` against each of the three chunking techniques and averaged the per-question results. The full run output is shown above. These same values were independently re-verified using `compute_metrics.py`, which recomputes this table purely from the saved raw/ CSV files, without re-running any embeddings.

Confidently scored retrieval that does not contain the answer:

For Q4, Token Chunking ranked a chunk from NCT02722239 as the third-best result. This study was about Dapagliflozin and Metformin compared with Forxiga in healthy volunteers, so it was unrelated to the question about Semaglutide and Ozempic. However, it received a similarity score of 0.5595, which was even higher than the relevant result’s score of 0.5355.

The model probably chose this chunk because it contained several words and ideas similar to the question, such as “Phase III,” “Metformin,” “combination,” a reference product, and a dosage. The text had a similar overall structure to the question, even though the drugs and

participants were completely different. This shows that the embedding model can recognize similar wording and patterns without fully understanding the specific topic.

Observations:

Sentence-window chunking gave the most consistently good retrievals overall. It had the highest mean@3 cosine score (0.676) and perfect recall (1.0), meaning it found the correct source document for every one of my five questions. This makes sense because sentence-window chunking searches at the level of a single, precise sentence. When a question has a very specific answer, like an exact dose or a named comparison, this technique can match that sentence directly instead of it being buried inside a bigger chunk. That said, the best performing technique was not the same for every individual question. Token chunking had the highest top-1 cosine for q1 (0.8224). Sentence-window won for q2 and q4. Semantic won for q3 and q5. This suggests no single chunking strategy is universally best. Which one wins seems to depend on how and where the specific answer is phrased within the source document.

Token chunking was the weakest overall. It missed the correct source document entirely for one question, q5, about oil palm phenolics. This gave it a recall@k of 0.80, compared to the other two techniques' perfect 1.0. I think this happened because token chunking cuts text every fixed number of tokens with no awareness of where one idea ends and another begins. So the answer may have ended up split awkwardly across chunk boundaries, or diluted by unrelated nearby text. Semantic chunking's larger average chunk size, 1650 characters compared to sentence-window's 210, also seemed to slightly dilute its similarity scores. This happened even when it retrieved the correct document, since each chunk carries more surrounding, less relevant text alongside the actual answer.

Conclusion:

Based on my measurements, I judge Sentence-window chunking to be the best technique for this corpus. It achieved perfect recall@k of 1.0. It also had the highest mean@3 cosine score of 0.676 among all three techniques. It consistently surfaced the exact sentence containing the answer, rather than a larger chunk that only partially matched. Token chunking was the fastest technique, and Semantic chunking also achieved perfect recall. However, Sentence-window's combination of precision and reliability makes it the strongest overall choice for this corpus. Individual clinical trial facts, like a specific dose, a specific comparison, or a specific eligibility number, tend to live in single, self-contained sentences. This is exactly what sentence-window chunking is built to capture. The main tradeoff is that Sentence-window produced far more chunks than the other two techniques, 1063 compared to 258 for Token and 135 for Semantic. It also had the highest average retrieval

latency, at 10.29 milliseconds compared to 4 to 6 milliseconds for the others. This is a reasonable cost given how much more consistently accurate it was.